

****Modèles Économiques et Open Source en Entreprise****

:::

Session 9 – Cours M1 Introduction aux Logiciels Libres

****Objectifs de la séance****

****Ce que vous saurez faire****

À la fin de cette séance, vous serez capables de :

- **Identifier** les principaux modèles économiques de l'open source
- **Analyser** les tensions entre communauté et business
- **Comprendre** le rôle d'un OSP0 et de l'InnerSource
- **Évaluer** les obligations de conformité d'une entreprise

****Partie 1******Le paradoxe de l'open source**

****Comment gagner de l'argent avec du "gratuit" ?****

| "If the software is free, how do you make money?"

Réponse courte : Le logiciel est libre, pas les services associés.

Réponse longue : Plusieurs modèles économiques viables existent, chacun avec ses avantages et compromis.

****Ce que "gratuit" signifie vraiment******Ce qui est gratuit**

- Le code source
- Le droit d'utiliser
- Le droit de modifier
- Le droit de redistribuer

Ce qui ne l'est pas forcément

- Le support
- L'expertise
- L'intégration
- Les garanties
- Les fonctionnalités premium

****L'économie de l'abondance****

Dans le monde physique : rareté → valeur Dans le monde logiciel : **abondance** (copie gratuite)

Où est la valeur ?

- Temps et expertise (toujours rares)
- Confiance et réputation
- Personnalisation et intégration
- Support et garanties

****Le coût marginal du logiciel est nul****

Coût de création d'un logiciel : élevé (développement, tests, documentation)

Coût de reproduction : **zéro** (copier un fichier ne coûte rien)

Coût de distribution : **quasi nul** (Internet)

Conséquences fondamentales :

- Le logiciel est un bien **non-rival** : mon usage ne diminue pas le vôtre
- Vendre des copies n'a de sens que grâce au **droit d'auteur** (exclusivité artificielle)
- L'open source supprime cette exclusivité → il faut vendre **autre chose** que la copie
- D'où les modèles économiques basés sur les services, le support, la confiance, l'expertise

C'est cette propriété économique qui rend l'open source possible – et qui oblige à repenser les modèles de revenus.

****Partie 2******Les modèles économiques**

****Vue d'ensemble des modèles****

Modèle	Description	Exemples
Dual Licensing	GPL + Commercial	MySQL, Qt
Open Core	Core libre + plugins payants	GitLab, Rudder, Centreon
Distributeur	Intégration + support	Red Hat, SUSE
Services	Consulting, formation, intégration	Smile, Oslandia
SaaS	Version hébergée	GitLab.com, Clever Cloud
Donations/Sponsors	Financement communautaire	curl, Vue.js

****Dual Licensing****

Principe : Le même code sous deux licences.

Licence copyleft (GPL)

- Gratuit
- Copyleft s'applique
- Pour les projets libres

Licence commerciale

- Payant
- Pas de copyleft
- Pour l'usage propriétaire

Exemples : MySQL, Qt, MongoDB (historique)

Limites : Nécessite le contrôle du copyright (CLA), perçu comme "exploitation".

■ Rappel : le mécanisme du dual licensing a été vu en sessions 5-6.

****Open Core****

Principe : Cœur libre + extensions propriétaires.

Enterprise Features
Core Features

← Propriétaire, payant

← Open source, gratuit

Exemples : GitLab (CE vs EE), Grafana, Rudder, Centreon

Le dilemme :

- Trop de features dans le core → pas de revenus
- Trop peu dans le core → communauté frustrée

****Distributeur / Support****

Principe : Vendre du support et de l'intégration.

Red Hat (modèle emblématique) :

- Linux et les logiciels sont libres
- Red Hat vend : support, certifications, intégration, SLA

Ce qui est gratuit

- Le code source (Fedora, CentOS Stream)
- Les binaires rebuilds (Rocky, Alma)

Ce qui est payant

- RHEL (binaires officiels)
- Support 24/7, certifications, garanties

Rachat par IBM (annoncé 2018, finalisé 2019) : 34 milliards \$

****Services professionnels : transversal à tous les modèles****

Quel que soit le modèle principal (open core, dual licensing, SaaS...), les **services à haute valeur ajoutée** sont une source de revenus majeure pour tous les éditeurs, open source ou non.

Types de services

- Consulting et architecture
- Développement sur mesure
- Formation et certification
- Intégration et migration
- Support et SLA

Pourquoi c'est important

- Revenu récurrent et prévisible
- Fidélisation des clients
- Remontée de besoins terrain
- Fonctionne même sans licence propriétaire

****Réputation et confiance : "Red Hat, c'est comme Evian"****

"Red Hat c'est comme Evian: L'eau est gratuite. Pourtant Evian vend des milliards de bouteilles. Pourquoi ? Parce que vous **faites confiance** à la marque." – Bob Young, ancien CEO de Red Hat (1998)

Ce que vend réellement un éditeur open source :

- **Garantie** : ça marche, et si ça casse, quelqu'un répond
- **Certification** : hardware, cloud, conformité (FIPS, Common Criteria ...)
- **Réputation** : "personne ne s'est fait virer pour avoir choisi Red Hat"
- **Responsabilité juridique** : indemnisation en cas de violation de licence

La certification est un levier particulièrement puissant : elle crée un écosystème (partenaires, compétences, emplois) qui renforce la position de l'éditeur.

****SaaS : le modèle et ses limites****

Principe : Héberger le logiciel et vendre l'accès comme service.

Avantages : Revenus récurrents, contrôle de l'expérience, scalabilité.

Exemples : GitLab.com, MongoDB Atlas, Clever Cloud

****Le "Cloud Problem"****

Le problème : La licence open source autorise **n'importe qui** à proposer le logiciel comme service – y compris les hyperscalers (AWS, GCP, Azure).

Scénario type :

1. Un éditeur crée une base de données open source (ex: Elasticsearch)
2. Il finance le développement, paie les développeurs, anime la communauté
3. AWS lance "**Amazon OpenSearch Service**" – le même logiciel, hébergé par AWS
4. AWS a plus de clients, plus de datacenters, plus de force commerciale
5. L'éditeur original perd des clients SaaS au profit d'AWS

Le paradoxe : La licence permet cela. C'est **légal**. Mais l'éditeur qui a tout financé se retrouve en concurrence avec un géant qui n'a pas contribué au développement.

■ "We built it, they sell it." – Sentiment partagé par MongoDB, Elastic, Redis...

****Les réponses au Cloud Problem******Réponse 1 : Changer de licence**

Licence	Restriction
SSPL (MongoDB)	Interdit d'offrir comme service
BSL (MariaDB)	Devient open source après délai
Elastic License	Restrictions cloud

Controverse : **pas open source** selon l'OSI.

Réponse 2 : Différenciation

- Features exclusives SaaS (GitLab)
- Expérience utilisateur supérieure
- Intégration, support, certification
- Marque et confiance

****Les licences "chrono-dégradables" (Delayed Open Source)****

Principe : Le code est propriétaire (ou réservé) pendant un temps, puis devient open source.

Modèle	Mécanisme de libération	Exemple
Ghostscript (années 1990)	Version N-1 libérée sous GPL quand la version N sort	Précurseur historique
BSL (Business Source License)	Conversion automatique en open source après un délai fixe (typiquement 3-4 ans)	MariaDB, HashiCorp
Sponsorware	Features libérées quand un seuil de sponsoring est atteint	Material for MkDocs

Material for MkDocs (squidfunk) : les features "Insiders" sont réservées aux sponsors GitHub. Elles sont libérées sous MIT non pas après un délai, mais quand un **niveau de financement mensuel** est atteint – les sponsors financent collectivement la libération.

Limites : Dépendance aux intermédiaires (GitHub Sponsors, Open Collective) qui prélèvent une commission et contrôlent la relation avec les sponsors.

****Donations et Sponsors****

Principe : Financement par la communauté/entreprises.

Plateformes

- GitHub Sponsors
- Open Collective
- Tidelift (enterprise)

Financement public

- NLNet (Europe)
- Sovereign Tech Fund (Allemagne)
- NGI (Next Generation Internet, UE)

Exemples réussis : Vue.js (Evan You), Babel, Godot Engine

Limite : Rarement suffisant pour vivre à plein temps, sauf pour les "superstars"

****L'écosystème open source français****

La France a un tissu d'éditeurs et d'intégrateurs open source significatif.

Entreprise	Modèle	Produit / Activité
Nexedi	Services + SaaS	ERP5, SlapOS
XWiki	Open core + SaaS	Wiki d'entreprise, CryptPad
Rudder	Open core	Automatisation IT / conformité
Centreon	Open core	Supervision informatique
Smile	Services / intégration	Plus grand intégrateur européen
Clever Cloud	SaaS (PaaS)	Hébergement cloud
BlueMind	Open core	Messagerie collaborative
Enalean	Open core + services	Tuleap (ALM)

Structuration : CNLL (Union des entreprises du logiciel libre), OSSFR (ex-April).

****Partie 3******Cas d'études et tensions**

****Cas 1 : Le changement de licence HashiCorp****

Août 2023 : HashiCorp change Terraform de MPL à BSL. **Avril 2024** : IBM rachète HashiCorp pour **6,4 milliards \$**.

Raison invoquée

- Concurrents "parasites"
- Notamment : cloud providers

Réaction communauté

- Fork → OpenTofu (Linux Foundation)
- Le rachat par IBM renforce le soupçon de "bait and switch" pré-acquisition

****Cas 2 : Redis et Valkey****

Mars 2024 : Redis Labs change la licence des modules.

Avant

- Redis (BSD)
- Modules (BSD → RSAL/SSPL)

Après

- Fork Valkey (Linux Foundation)
- Soutenu par AWS, Google, Oracle

****La tension communauté / business******Intérêts de la communauté**

- Tout devrait être libre
- Gouvernance ouverte
- Pas de "faux" open source

Intérêts business

- Revenus durables
- Protection contre les parasites
- Contrôle stratégique

Équilibre difficile à trouver et maintenir.

****Rappel : Comment fonctionne le Venture Capital****

Le VC (Venture Capital / Capital-risque) finance des startups en échange de parts du capital.

Mécanisme simplifié :

1. La startup a une **valorisation** (ex: 10 M€ "pre-money")
2. Le VC investit (ex: 5 M€) → valorisation "post-money" = 15 M€, VC détient 33%
3. Le VC a des **actions de préférence** (liquidation prioritaire, anti-dilution)
4. L'objectif du VC : un **exit** (revente ou IPO) avec un **multiple d'au moins x10** en 5-7 ans

Conséquence pour l'open source : Le VC n'investit pas par philanthropie. Il exige une croissance agressive et un exit lucratif. Si le modèle open source ne génère pas assez de revenus → pression pour changer la licence ou se faire racheter.

****Le "Bait and Switch"****

Pattern récurrent chez les éditeurs VC-funded :

1. Lancer un projet open source → adoption rapide, communauté gratuite
2. Lever des fonds (VC) sur la base de cette adoption
3. Construire un produit commercial autour du projet
4. Changer la licence une fois établi (SSPL, BSL, Elastic License...)
5. Monétiser la communauté captive – ou se faire racheter (exit pour le VC)

Exemples : MongoDB (IPO), Elastic (IPO), HashiCorp (racheté IBM), Redis, CockroachDB, Sentry ...

****Cas 3 : WordPress vs WP Engine (2024)****

Les faits : Matt Mullenweg (BDFL WordPress, CEO Automattic) reproche à WP Engine – hébergeur WordPress majeur (valorisé 3 Md\$) – de ne pas contribuer suffisamment au projet.

L'argument de Mullenweg

- WP Engine profite de l'écosystème
- Contributeurs : 0 à la core team
- Ne contribue pas financièrement à la fondation

L'argument de WP Engine

- Contribue via plugins et écosystème
- Les moyens de pression sont disproportionnés
- Confusion entre fondation et intérêts d'Automattic

Question soulevée : Quand une seule personne contrôle le projet **et** l'entreprise commerciale, qui arbitre ?

****Signes d'alerte******Projet potentiellement à risque**

- Single-vendor (une seule entreprise)
- CLA exigeant (copyright assignment)
- Financement VC important
- Pas de gouvernance communautaire

Projet plus sûr

- Multi-vendor
- Fondation neutre
- DCO simple
- Gouvernance transparente

****VC-funded vs bootstrapped****

Le mode de financement influence fortement la trajectoire d'un éditeur open source.

VC-funded → pression de sortie

- Croissance rapide exigée
- Exit (IPO ou rachat) attendu
- Changement de licence fréquent
- Ex: MongoDB, HashiCorp, Elastic

Bootstrapped → stabilité

- Croissance organique
- Pas de pression actionnariale
- Licence stable dans le temps
- Ex: XWiki, Nexedi, Enalean (Tuleap)

Les projets communautaires (PostgreSQL, Linux) évitent la pression capitaliste, mais ont d'autres risques : désengagement des contributeurs faute de modèle économique (Apache Attic, Apache OpenOffice après le retrait d'IBM), ou conflits fondation vs éditeur principal (TDF vs Collabora, cf. session 8).

****Open source en entreprise****

Face à ces enjeux – modèles économiques, tensions communautaires, risques de conformité – les entreprises doivent **s'organiser**.

Trois réponses complémentaires :

1. **Conformité** – Respecter les obligations légales des licences
2. **OSPO** – Gouverner l'usage et la contribution open source
3. **InnerSource** – Appliquer les pratiques open source en interne

****Partie 4****

Conformité

****Obligations légales liées aux licences****

■ Rappel des sessions 4-5 – ici dans le contexte de la gestion en entreprise.

Toute entreprise utilisant de l'open source doit :

Licences permissives

- Conserver les notices de copyright
- Inclure la licence
- Parfois : disclaimer de garantie

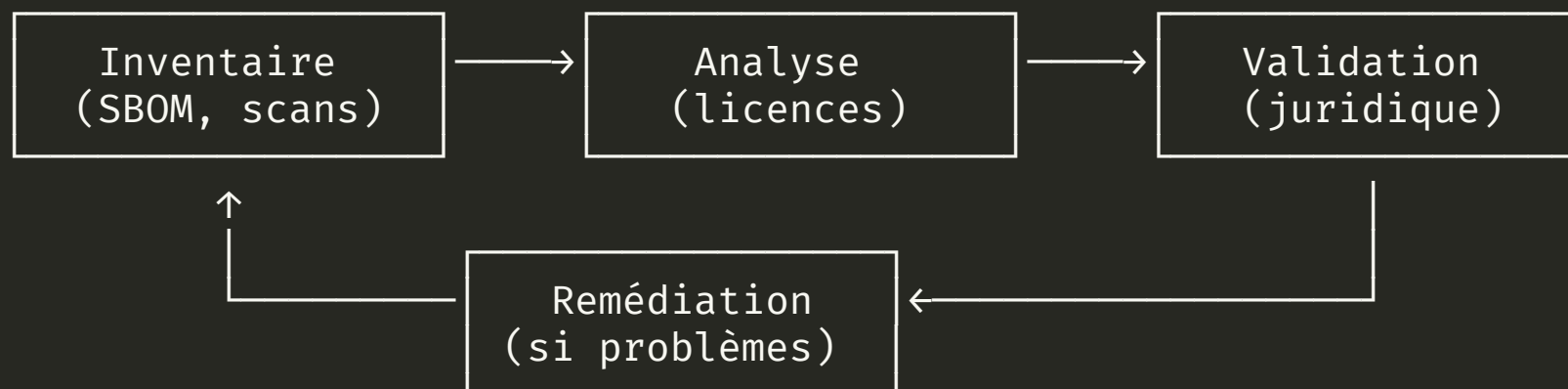
Licences copyleft

- Tout ce qui précède
 - Fournir le code source modifié
 - Utiliser la même licence

****Risques de non-conformité******Cas réels :**

Entreprise	Violation	Conséquence
Cisco (2008)	GPL dans Linksys	Procès FSF, settlement
Free (2011)	GPL Freebox	Condamné (France), code source publié
VMware (2015)	Linux kernel	Procès, abandonné
Hancom (2016)	GPL Ghostscript	Condamné (Corée)
SFC vs Vizio (2021)	GPL	Settlement 2025
Orange (2011→2024)	GPL v2 (lib Lasso)	Condamné à 800 k€ (France)

****Processus et outils de conformité****



Outils :

Outil	Type	Fonctionnalités
REUSE (FSFE)	Open source	Vérification des métadonnées licence/copyright par fichier
FOSSology	Open source	Scan, identification, reporting
ScanCode	Open source	Détection de licences
Black Duck	Commercial	Enterprise, compliance
FOSSA	Commercial	CI/CD integration

****Partie 5****

L'OSPO (Open Source Program Office)



****Qu'est-ce qu'un OSP0 ?****

Open Source Program Office – Structure dédiée à la gestion de l'open source en entreprise.

Missions principales

- Politique d'utilisation
- Conformité licences
- Contribution upstream
- Relations communautaires

Rattachement

- CTO / Direction technique
- Direction juridique
- Parfois indépendant
- Transverse aux équipes

****Pourquoi créer un OSP0 ?****

Enjeux pour l'entreprise :

1. **Juridiques** – Conformité aux licences, gestion des risques
2. **Sécurité** – Supply chain, vulnérabilités
3. **Stratégiques** – Influence, recrutement, innovation
4. **Économiques** – Optimisation des coûts, éviter le vendor lock-in

■ Les entreprises utilisent en moyenne 500+ composants open source.

****Les responsabilités d'un OSP0******Consommation**

- Inventaire des dépendances
- Validation des licences
- Gestion des vulnérabilités
- Formation des développeurs

Contribution

- Politique de contribution
- Process de validation
- Relations avec les projets
- Sponsoring / donations

****Entreprises avec un OSP0****

Entreprise	OSP0 depuis	Focus
Google	2004	Innovation, Android, Kubernetes
Microsoft	2014	Transformation, GitHub
Red Hat	Origine	Core business
SAP	2017	Conformité, contribution
Spotify	2018	InnerSource, outils
SNCF	2020	Souveraineté

TODO Group (Linux Foundation) : communauté de 80+ OSP0 – <https://todogroup.org/>

****Partie 6******InnerSource**

****Qu'est-ce que l'InnerSource ?****

Appliquer les pratiques open source à l'intérieur de l'entreprise.

Open Source

- Code public
- Contributeurs externes
- Communauté mondiale
- Gouvernance ouverte

InnerSource

- Code interne partagé
- Contributeurs = employés
- Communauté entreprise
- Gouvernance adaptée

****Problèmes résolus par l'InnerSource****

Situation typique :

- Équipe A développe un composant
- Équipe B a besoin d'une modification
- Équipe A est occupée → ticket dans le backlog
- Équipe B attend... ou duplique le code

Avec InnerSource :

- Équipe B propose une PR
- Équipe A review et merge
- Tout le monde gagne du temps

****Rôles et mise en place****

Rôle	Responsabilités
Guest	Contributeur externe à l'équipe
Trusted Committer	Mainteneur, review, mentoring
Product Owner	Vision, priorités, roadmap
Contributor	Développeur qui soumet des PRs

Étapes recommandées :

1. **Pilote** – Commencer avec 2-3 projets volontaires
2. **Outillage** – Forge interne (GitLab, GitHub Enterprise)
3. **Culture** – Valoriser les contributions cross-équipes
4. **Métriques** – Mesurer l'adoption et les bénéfices

InnerSource Commons : patterns et bonnes pratiques – <https://innersourcecommons.org/>

****Résumé****

****Ce qu'il faut retenir****

1. **Coût marginal nul** → l'open source oblige à vendre autre chose que la copie
2. **Plusieurs modèles** viables : services, open core, dual licensing, SaaS
3. **Réputation et certification** : la confiance est le vrai produit
4. **Conformité** : obligations réelles, risques juridiques, outils disponibles
5. **OSPO** : structure clé pour gouverner l'open source en entreprise
6. **InnerSource** : appliquer les pratiques open source en interne

****Pour la prochaine séance****

Séance 10 : Supply chain et sécurité

Préparation suggérée :

- Qu'est-ce que Log4Shell ?
- Qu'est-ce qu'un SBOM ?

****Questions ?****

:::

Passons au TD